

INSTITUTE OF ENGINEERING AND MANAGEMENT, KOLKATA

Assignment 7

Page Object Model (POM) Restructure

Name:	Ashmita Saha
Enrollment no.:	12023002028118
Department:	CSE (AI & ML)
Course:	Python for Automation
Academic Session:	2023-2027
Submission Date:	18 Sept, 2026

1. Module Name & Experiment Title

Module Name	Advanced Test Automation Frameworks & Architectural Patterns
Experiment Title	Page Object Model (POM) Restructure for Selenium Test Automation Suite

2. Problem Statement

Take your working functional test scripts from Tier 1 and restructure them into a Page Object Model (POM) design pattern. Create separate page classes (e.g., LoginPage, DashboardPage) containing only locator definitions and UI methods. Keep your actual test assertions completely separate from the locators.

3. Objective

- To implement the Page Object Model (POM) architectural design pattern in Selenium using Python.
- To encapsulate web element locators and UI interaction methods within specific page classes (LoginPage and DashboardPage).
- To decouple test assertions completely into a dedicated execution script (test_login.py) to increase test modularity and maintainability.
- To execute automated user authentication against `https://www.saucedemo.com/` and validate post-login URL and dashboard title assertions.

4. Tools, Software, and Concepts Used

Programming Language	Python 3.x
Automation Framework	Selenium WebDriver
Web Browser & Driver	Google Chrome & ChromeDriver
Design Pattern	Page Object Model (POM) Architecture
Key Concepts	Object-Oriented Programming (OOP), Encapsulated Locators (By.ID, By.NAME, By.XPATH, By.CLASS_NAME), Tuple Unpacking (*locator), Assertions Separation.

5. Source Code & Implementation Steps

login_page.py

The LoginPage class defines element locators for the login credentials and submit action, alongside UI methods for performing user login.

```

from selenium.webdriver.common.by import By

class LoginPage:
    username = (By.ID, "user-name")
    password = (By.NAME, "password")
    login_button = (By.XPATH, "//input[@id='login-button']")

    def __init__(self, driver):
        self.driver = driver

    def enter_username(self, username):
        self.driver.find_element(*self.username).send_keys(username)

    def enter_password(self, password):
        self.driver.find_element(*self.password).send_keys(password)

    def click_login(self):
        self.driver.find_element(*self.login_button).click()

    def login(self, username, password):
        self.enter_username(username)
        self.enter_password(password)
        self.click_login()

```

Figure 5.1: Source code screenshot of login_page.py

dashboard_page.py

The DashboardPage class encapsulates locators and helper functions to fetch dashboard properties such as current URL and inventory page title after successful login.

```

from selenium.webdriver.common.by import By

class DashboardPage:
    inventory_title = (By.CLASS_NAME, "title")

    def __init__(self, driver):
        self.driver = driver

    def get_url(self):
        return self.driver.current_url

    def get_title(self):
        return self.driver.find_element(*self.inventory_title).text

```

Figure 5.2: Source code screenshot of dashboard_page.py

test_login.py

The test_login.py script initializes the Chrome driver, instantiates the page objects, performs actions via the POM interface, and executes assertions independently from locators.

```
from selenium import webdriver
from login_page import LoginPage
from dashboard_page import DashboardPage

driver = webdriver.Chrome()
driver.get("https://www.saucedemo.com/")
driver.maximize_window()

login_page = LoginPage(driver)
dashboard_page = DashboardPage(driver)

login_page.login("standard_user", "secret_sauce")

assert "/inventory.html" in dashboard_page.get_url()
assert dashboard_page.get_title() == "Products"

print("Login successful!")
print("Dashboard URL:", dashboard_page.get_url())

driver.quit()
```

Figure 5.3: Test script implementation screenshot of test_login.py

6. Output & Console Log

Execution of test_login.py launches Chrome, authenticates the standard user, confirms assertions on the dashboard, prints confirmation output, and gracefully quits the driver instance.

```
$ python test_login.py
DevTools listening on ws://127.0.0.1:58492/devtools/browser/...
Login successful!
Dashboard URL: https://www.saucedemo.com/inventory.html
Process finished with exit code 0
```

7. Brief Result, Observation, and Conclusion

Result: The Tier 1 automation scripts were successfully refactored into the Page Object Model (POM) structure. The test script executed without errors, meeting all locator separation and assertion criteria.

Observation:

- By storing web element locators inside class attributes as tuples (e.g., `By . ID`, `By . XPATH`), UI structural changes can be updated in a single file without modifying test logic.
- The test script remains clean, highly readable, and focused solely on test workflow execution and assertion validation.
- Reusability is significantly improved as methods like `login ( )` can be seamlessly reused across multiple test suites.

Conclusion: Structuring test automation suites with the Page Object Model enhances code maintainability, reduces duplication, and establishes a robust framework architecture suitable for enterprise-grade automation testing.